Autonomous Navigation in Dynamic Environments via PPO

Jovan Tan[1]

Abstract
This project evaluates the efficiency of Proximal Policy Optimization (PPO) in developing autonomous navigation capabilities within a physics-based Unity simulation. By employing the ML-Agents framework, a car agent was tasked with navigating a closed-loop track featuring dynamic and randomized obstacles. The study specifically investigates the transition from sparse to dense reward structures to overcome the credit assignment problem and mitigate "lazy" agent behaviors, such as intentional self-termination. Results show that a combination of hierarchical reward shaping by balancing survival incentives with checkpoint milestones, and curriculum based obstacle randomization, significantly improves policy generalization. The final model successfully transitioned from memorizing static paths to reactive obstacle avoidance, achieving near-optimal lap completion rates across varied obstacle configurations.

Authors
[1] *Media Technology Student at Linköping University, jovta848@student.liu.se*

Keywords: Reinforcment Learning — PPO — Deep Neural Network — Reward Shaping

Contents

1 Introduction 1
2 Theory 1
2.1 Proximal Policy Optimization (PPO) 2
2.2 Reward Shaping 2
2.3 Observation and Action Spaces 2
3 Method 2
3.1 Environment Design 2
Track and Checkpoint System • Obstacle Randomization • Parallel Training Setup
3.2 Agent Architecture 2
Observation Space • Action Space • Neural Network Architecture
3.3 Reward Function Design 3
Positive Rewards
3.4 Training Configuration 3
4 Result 4
4.1 Training Progress 4
5 Discussion 4
5.1 Reward Engineering: The Design Challenge . . . 4
The Immediate Crash Phenomenon • Dense vs. Sparse Reward Balancing • Generalization Through Randomization
6 Conclusion 5
Acknowledgments 5
References 5

1. Introduction

Autonomous navigation in dynamic environments remains a difficult problem due to unpredictable obstacles that might be in the way. While traditional heuristic path planning often lacks flexibility, Deep Reinforcement Learning (DRL) offers a data-driven alternative capable of learning generalizable policies.

This project investigates the viability of DRL within a simple physics simulation, utilizing the Unity engine [1] and the ML-Agents framework [2]. The primary research objective is to evaluate whether Proximal Policy Optimization (PPO) can successfully train an agent (car) to navigate tracks containing randomized obstacles.

2. Theory

At its core, Reinforcement Learning (RL) is a cycle of **agent-environment interaction**. This relationship is built upon what is called the Markov Decision Process (MDP), a mathematical framework used for modeling sequential decision making in uncertain environments, where an agent chooses actions to maximize cumulative rewards over time. The Markov property states that the future depends only on the present state, not the entire past history [3].

In ML-Agents, the agent develops a **policy**, which is a mapping from states to the best possible actions. While value based methods focus on predicting the total reward of a state, ML-Agents use **Actor-Critic** architecture that simultaneously

learns to predict rewards (Critic) and execute Actions (Actor) to improve training stability.

2.1 Proximal Policy Optimization (PPO)

PPO is the industry standard algorithm for Unity ML-Agents due to its balance of ease of tuning and training stability. The key mechanisms and hyperparameters are as follows:

- **Clipped Surrogate Objective:** PPO uses a "clipping" mechanism (ε) that restricts the size of policy updates, ensuring the agent's behavior does not change too drastically in one step.
- **Beta β (Entropy Regularization):** This hyperparameter encourages exploration by rewarding the agent for keeping its actions varied, preventing it from settling on a "safe" but mediocre path too early in training.
- **Learning Rate and Buffer Size:** These manage how much the agent learns from each batch of experience. In ML-Agents, larger buffers often lead to more stable updates at the cost of slower training times.

2.2 Reward Shaping

Reward shaping is the process of adding intermediate rewards to guide an agent toward a goal, effectively solving the **credit assignment problem** - the difficulty of identifying which past actions led to a current success or failure. These rewards can be given in different methods, **Dense** and **Sparse**. While a sparse reward (e.g., +100 for finishing a lap) is the ultimate goal, it is often too rare for an agent to find randomly [4]. We use **dense rewards**, such as small increments for maintaining velocity, going through checkpoints, and staying alive to provide constant feedback in order for it to better know how its actions are affecting the outcome.

2.3 Observation and Action Spaces

To navigate dynamic environments and not blindly learn the timings to turn, the agent requires a perception system and a method for precise control.

- **Ray Perception Sensors:** These simulate LiDAR by casting rays at set angles and returning the distance and type (tag) of any object hit. This provides a lightweight vector-based "vision".
- **Continuous Control:** Unlike discrete systems (e.g., "turn left" or "right"), autonomous driving requires a **continuous action space**. The agent outputs decimal values for steering and throttle, allowing for the smooth and high-precision maneuvers necessary for high-speed obstacle avoidance.

3. Method

This section describes the implementation of an autonomous driving agent using RL within the Unity ML-Agents frameowrk. The method ecompasses environment design, agent architecture, reward function engineering, and training configuration.

3.1 Environment Design

The training environment was implemented in Unity 2022. The simulation consists of a closed-loop racing track with checkpoint-based navigation system and dynamic obstacles design to evaluate collision avoidance capabilities.

3.1.1 Track and Checkpoint System

The track features a sequential checkpoint system where each checkpoint is assigned an index (0, 1, 2, ..., N). The agent must traverse checkpoints in the correct order to receive rewards. A lap is considered complete when the agent passes through checkpoint 0 (which is the start and finish line) after having successfully visited all other checkpoints in sequence. The checkpoint system was implemented using trigger colliders that detect when the car agent passes through each checkpoint.

3.1.2 Obstacle Randomization

To prevent overfitting and encourage generalization, dynamic obstacles were implemented with randomization. Each obstacle, controlled by the *MovingObstacle* class, can operate in three modes.

- **Static Non-Random:** Stationary obstacles for initial learning of what an obstacle is.
- **Static Randomization:** At the start of each episode, obstacles randomly spawn at either their original position or at a predefined offset, with a 50% probability for each configuration. This simulates opening either the left or right side of the track.
- **Dynamic movement:** Obstacles continuously interpolate between their original and offset positions using linear interpolation (lerp) at configurable speeds, creating moving hazards that require more planning from the agent.

These different modes are crucial in order to follow a curriculum based learning method, whereby you increase the difficulty of the task.

3.1.3 Parallel Training Setup

To accelerate training, the environment was duplicated to run four identical tracks in parallel, each with its own independent agent and unique track setup. This increases sample collection efficiency by 4, allowing for faster policy convergence as well as increased generalization.

3.2 Agent Architecture

The car agent was implemented as a subclass of Unity ML-Agents *Agent* base class, enabling integration with the PPO training algorithm ML-Agent provides.

3.2.1 Observation Space

The agent's observation space are the different variables it is aware of. For this agent, they consist of two vector observations and two sets of ray perception sensors.

- **Vector Observations:** It was aware of its current velocity as well as lap progress.
- **Ray Perception Sensors:** Detecting track boundaries (the walls) as well as the obstacles for collision avoidance separately.

Both sensors cast multiple rays in a radial pattern around the agent, with configurable ray length and detection tags. This separation allows for distinct reward shaping based on the detected object type.

3.2.2 Action Space

As mentioned above, the agent operates in a continuous action space in order to make smooth maneuvers, these consists of two dimensions:

- **Action[0] - Steering:** Continuous value in range [-1, 1], where -1 represents full left turn, 0 is straight, and +1 is full right turn. The turning rate is 300 degrees per second.
- **Action [1] - Acceleration:** Continuous value in range [0, 1] to prevent reverse movement. Values represent throttle intensity, with maximum forward speed set to 10 units per second.

The car's movement model includes realistic physics constraints: steering is only permitted when velocity exceeds 0.1 units per second, and a drag deceleration of 8 units/s^2 is applied when not actively accelerating. Position updates use Unity's *Rigidbody.MovePosition()* for physics-based movement, while rotation uses *Transform.Rotate()* for steering control.

3.2.3 Neural Network Architecture

The policy network consists of a feedforward neural network with the following architecture:

- **Input layer:** Observation vector (2 scalar observations + ray sensor input)
- **Hidden layers:** 2 fully connected layers with 128 units each
- **Output layer:** 2 continuous actions (steering and acceleration)
- **Activation function:** ReLU (unbounded positive values)
- **Normalization:** Disabled to preserve raw observation magnitudes

3.3 Reward Function Design

The reward function underwent multiple iterations to address emergent unwanted behaviors (discussed more in chapter 5). The final reward structure balances multiple objectives through carefully tuned reward components:

3.3.1 Positive Rewards

- **Checkpoint Reward:** +50 points for passing through the correct checkpoint in sequence. This provides clear milestone feedback for successful navigation.
- **Goal Reward:** +100 points for completing a full lap. This serves as the primary episode (attempt) success signal hence the large amount of points.
- **Progress Reward:** +0.01 $\times \Delta d$ per time frame, where Δd is the decrease in distance to the next checkpoint. This dense reward signal guides the agent toward the next checkpoint instead of blindly looking for one.
- **Survival reward:** +0.001 points per frame for continued operation. This encourages longer episodes (staying alive) and discourages early self-termination (discussed in 5).
- **Safety Distance Reward:** +0.01 $\times (d/d_max)$ points per frame when obstacles are detected beyond the proximity threshold (4.0 units), where d is the obstacles distance and d_max is the maximum ray length. This rewards obstacle awareness.

There were two scenarios where the agent would be penalized, wall collision and obstacle collision. Both result in a deduction of 10 points as well as immediate episode termination.

3.4 Training Configuration

Training was conducted using the Proximal Policy Optimization (PPO) algorithm with hyperparameter specified below:

PPO Hyperparameters:

- Learning rate: 0.0003 with linear decay schedule
- Batch size: 1024 samples
- Buffer size: 10240 experiences
- Epoch per update: 3
- Discount factor (γ): 0.99
- GAE parameter (λ): 0.95
- Clip range (ε): 0.2
- Entropy coefficient (β): 0.005

These were not specifically chosen but used as it was the preset standard for ML-Agents.

Training Protocol:

- Maximum training steps: 500,000
- Time horizon: 64 steps
- Summary Frequency: 10,000 steps
- Episode termination conditions: wall collision, obstacle collision, or lap completion

These training protocol parameters were set depending on how long the training was going to take. Multiple runs were conducted with variations in configuration parameters. Some runs were based on previous runs for the curriculum based method of learning while others were simply for different starting parameters.

4. Result

The results are shown below, presenting the different outcomes based on obstacles covering 70% of the track.

4.1 Training Progress

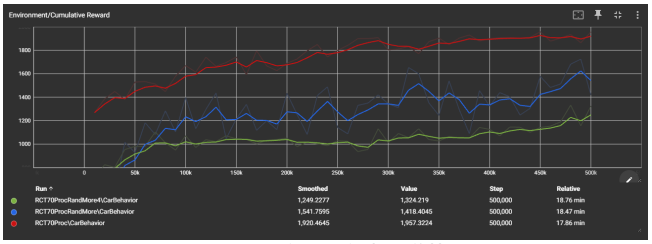

Figure 1. Reward graph for different runs

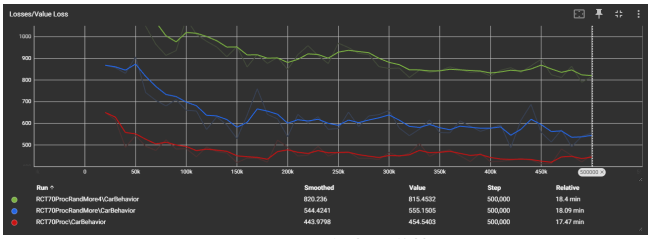

Figure 2. Loss graph for different runs

The above shown graphs, Figure 1 and Figure 2, show three different runs:

- **Red** - First initial run with one obstacle per straight track and no randomization for position.
- **Blue** - One of the middle runs (run 4 out of 7), has random obstacle positions as well as additional obstacles compared to the first run.
- **Green** - Last and latest run, three obstacles were added per straight track with randomized positioning.

Run	Final Reward Value	Loss
Run 1 (Red)	1920	444
Run 2 (Blue)	1542	544
Run 3 (Green)	1249	820

Table 1. Table of the values shown in Figure 1 and Figure 2. In comparison, all cars completing their laps results in a Reward Value of 1957 points with this projects settings.

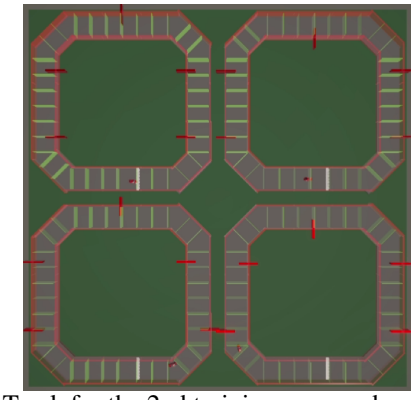

Figure 3. Track for the 2nd training run, random obstacles positioning, added number of obstacles

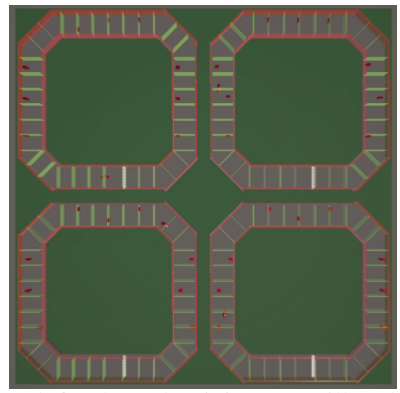

Figure 4. Track for the 3rd training run, pillar obstacles, randomized position, additional obstacles

5. Discussion

5.1 Reward Engineering: The Design Challenge

The development of the autonomous driving agent required several major iterations of reward function design, each revealing fundamental principles about RL specification and behavior shaping.

5.1.1 The Immediate Crash Phenomenon

Initial time-based penalties (negative rewards per frame) led to an unexpected optimal policy: **intentionally crashing immediately after episode start**. This was mathematically rational - early termination minimized accumulated penalties when per-frame costs exceeded collision penalties over typical episode duration.

This shows a fundamental RL challenge, that agents optimize the specified function, not the intended objective. The solution was to replace time penalties with survival rewards (+0.001/frame) which demonstrates a key principle, **reinforce desired states rather than punish undesired states**, reducing incentives for unexpected termination strategies.

5.1.2 Dense vs. Sparse Reward Balancing

Iteration three introduced survival rewards but progress rewards (+0.1/frame) accumulated so rapidly they dominated all

signals, causing aggressive maneuvering prioritizing instant progress over safety.

Iteration four implemented **hierarchical reward structure** through re-balancing of the rewards. Progress rewards gave 0.1, checkpoints gave 50 and collisions gave -10 points. This created distinct value gradients guiding better behavior.

5.1.3 Generalization Through Randomization

The final iteration introduced obstacle randomization (50% probability of offset positioning). This forced development of robust perception-to-action mappings rather than trajectory memorization, addressing **overfitting to environment specifics**. An issue with having obstacles non-randomized and too small giving the car a higher chance of completing a lap by doing something random gave high chance of overfitting as seen by one of the attempts made. One of the tracks managed to find a path that worked specifically for it and thus every other car agent would try to drive more like it due to having the same policy. However, since their tracks obstacles had other positioning, they would not be able to follow the same path as the one that succeeded. The car continuously completing, giving his action a high score while all the three other cars crashing giving them a low score was a clashing overfitting situation that only got resolved through making sure that there were more obvious paths to take (by blocking 50% of the road). This combined with randomizing after it has learned to pass a 50% blocked path led to better generalization.

6. Conclusion

This investigation confirms that while PPO provides a stable foundation for self learning autonomous car agents, the primary determinant of agent success is the architecture of the reward function. The iterative development process revealed that agents are highly sensitive to unintended mathematical incentives, specifically improper penalty weighting can lead to strategies like intentional crashing at start.

By pivoting to a dense, survival-oriented rewards structure and incorporating obstacle randomization, the agent moved beyond path memorization and toward genuine perception-based navigation. Future work should explore more complex sensors, such as integrating depth data with vector observations. This could, for example, make the agent aware of the velocity of different moving obstacles rather than their current position at each perception check.

Acknowledgments

I would like to thank Logan Shaffer, a fellow student, for the valuable discussions and collaborative brainstorming that helped define the strategy for model generalization and obstacle randomization.

References

[1] Arthur Juliani, Vincent-Pierre Berges, Ervin Teng, Andrew Cohen, Jonathan Harper, Chris Elion, Chris Goy, Yuan Gao, Hunter Henry, Marwan Mattar, and Danny Lange. Unity: A general platform for intelligent agents, 2020.

[2] Unity Technologies. Ml-agents toolkit documentation. https://github.com/Unity-Technologies/ml-agents, 2024. Accessed: 2026-01-20.

[3] Francisco M. Csobrino. Reinforcement learning: A practical guide to proximal policy optimization (PPO). https://medium.com/@csobrinofm/reinforcement-learning-a-practical-guide-to-proximal-policy-optimization-ppo-276df3e5099e, 2023. Accessed: 2026-01-20.

[4] Tim Miller. Reward shaping — Mastering Reinforcement Learning. https://gibberblot.github.io/rl-notes/single-agent/reward-shaping.html, 2019. Accessed: 2026-01-20.